Source Code Screenshots

Selected implementation excerpts demonstrating authentication, validation, authorization, file handling, React hooks/routing, and Flutter API/submission behavior.

The complete editable source accompanies these screenshots. Secrets and environment configuration are excluded.

1. Administrator authorization and validated management

server/src/admin.js

server/src/admin.js

Administrator authorization and validated management

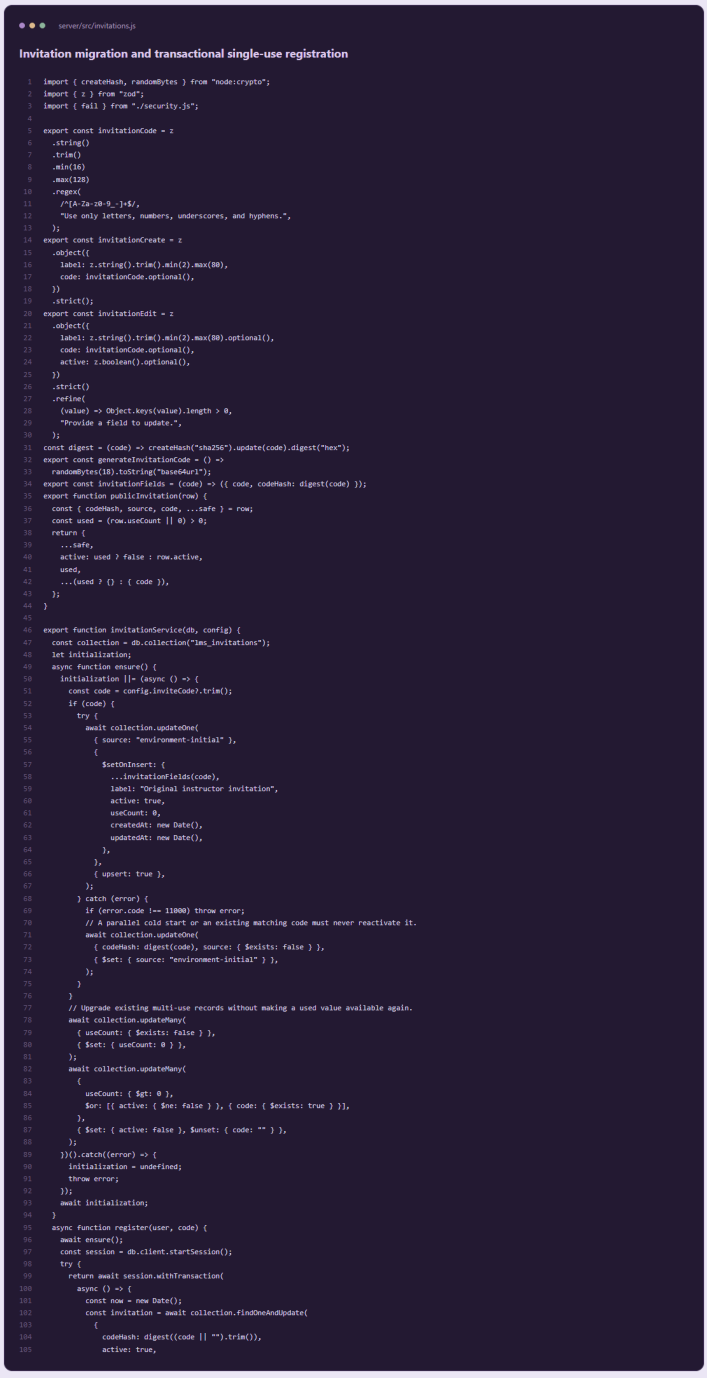
```
import { Router } from "express";
import bcrypt from "bcryptjs";
import { z } from "zod";
import { fail, id, publicUser, role } from "./security.js";
import { schemas, validate } from "./validation.js";
import {
  generateInvitationCode,
  invitationCreate,
  invitationEdit,
  invitationFields,
  publicInvitation,
} from "./invitations.js";

const activeCourses = { deletedAt: { $exists: false } };
const userProjection = {
  name: 1,
  email: 1,
  role: 1,
  status: 1,
  bio: 1,
  department: 1,
  createdAt: 1,
  updatedAt: 1,
  lastLoginAt: 1,
};
const editUser = z
  .object({
    name: schemas.register.shape.name.optional(),
    email: schemas.register.shape.email.optional(),
    department: z.string().trim().max(100).optional(),
    role: z.enum(["student", "instructor"]).optional(),
    status: z.enum(["active", "suspended"]).optional(),
  })
  .strict()
  .refine((v) => Object.keys(v).length > 0, "Provide a field to update.");
const createUser = schemas.register
  .omit({ inviteCode: true })
  .extend({ department: z.string().trim().max(100).default("") })
  .strict();
const listing = z
  .object({
    page: z.coerce.number().int().min(1).max(100000).default(1),
    search: z.string().trim().max(100).default(""),
    role: z.enum(["all", "student", "instructor", "admin"]).default("all"),
    status: z
      .enum(["all", "active", "suspended", "published", "draft", "archived"])
      .default("all"),
  })
  .strict();
const safeSearch = (value) => value.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
const account = (user) => ({
  ...publicUser(user),
  status: user.status || "active",
});
async function pageOf(collection, filter, page, projection) {
  const limit = 12;
  const [total, items] = await Promise.all([
    collection.countDocuments(filter),
    collection
      .find(filter, { projection })
      .sort({ createdAt: -1, _id: -1 })
      .skip((page - 1) * limit)
      .limit(limit)
      .toArray(),
  ]);
  return { items, total, page, pages: Math.ceil(total / limit) };
}

export function adminRoutes(db, invitations) {
  const router = Router();
  router.use("/admin", role("admin"));
  const users = db.collection("lms_users");
  const courses = db.collection("lms_courses");
  const activity = db.collection("lms_admin_activity");
  async function record(actor, action, target, detail) {
    await activity.insertOne({
      actorId: actor._id,
      actorName: actor.name,
      action,
      targetId: target._id,
      targetName: target.name || target.title,
      detail,
      createdAt: new Date(),
    });
  }
```

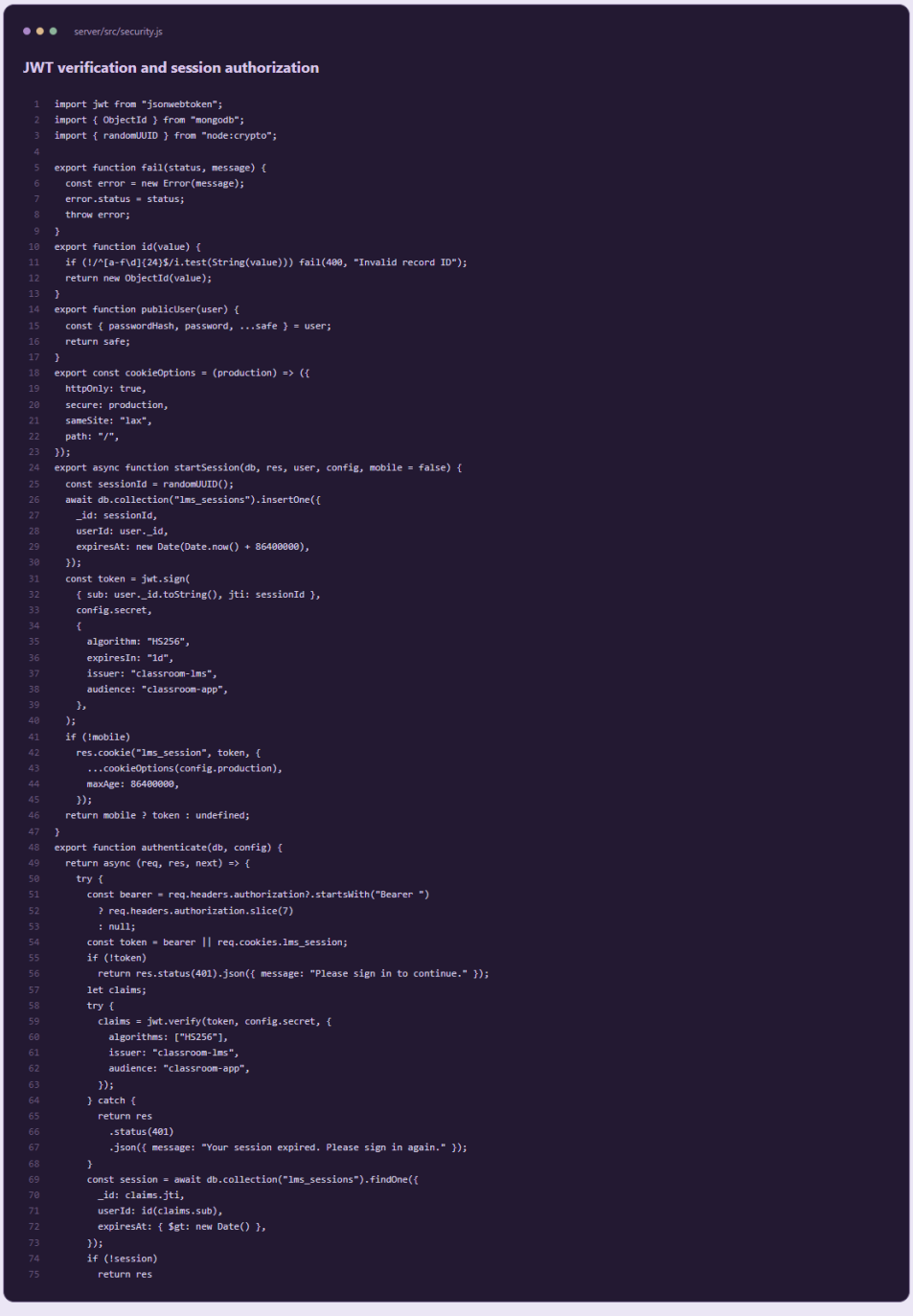
2. Invitation migration and transactional single-use registration

server/src/invitations.js



3. JWT verification and session authorization

server/src/security.js



4. Server input schemas and validation

server/src/validation.js

server/src/validation.js

Server input schemas and validation

```
import { z } from "zod";

const text = (min, max) => z.string().trim().min(min).max(max);
const password = z
  .string()
  .min(8)
  .max(72)
  .refine(
    (v) => Buffer.byteLength(v, "utf8") <= 72,
    "Password must be at most 72 UTF-8 bytes",
  )
  .regex(/[a-z]/, "Include a lowercase letter")
  .regex(/[A-Z]/, "Include an uppercase letter")
  .regex(/[0-9]/, "Include a number");
export const schemas = {
  register: z.object({
    name: text(2, 80),
    email: z
      .string()
      .trim()
      .email()
      .max(160)
      .transform((v) => v.toLowerCase()),
    password,
    role: z.enum(["student", "instructor"]).default("student"),
    inviteCode: z.string().max(200).optional(),
  }),
  login: z.object({
    email: z
      .string()
      .trim()
      .email()
      .max(160)
      .transform((v) => v.toLowerCase()),
    password: z.string().min(1).max(200),
  }),
  profile: z.object({
    name: text(2, 80),
    bio: text(0, 600).default(""),
    department: text(0, 100).default(""),
  }),
  password: z.object({
    currentPassword: z.string().min(1).max(200),
    newPassword: password,
  }),
  course: z.object({
    title: text(3, 120),
    code: text(3, 20)
      .regex(/^[a-zA-Z0-9 -]+$/)
      .transform((v) => v.toUpperCase()),
    description: text(10, 3000),
    category: text(2, 60),
    color: z
      .enum(["violet", "blue", "teal", "orange", "pink"])
      .default("violet"),
    published: z.boolean().default(false),
  }),
  assignment: z.object({
    title: text(3, 160),
    instructions: text(10, 6000),
    dueDate: z
      .string()
      .datetime({ offset: true })
      .transform((v) => new Date(v)),
    maxPoints: z.coerce.number().int().min(1).max(1000),
    allowLate: z.boolean().default(false),
  }),
  material: z.object({
    title: text(3, 160),
    description: text(0, 3000).default(""),
    url: z
      .union([
        z.literal(""),
        z
```

5. Course ownership and enrollment checks

server/src/courses.js

server/src/courses.js

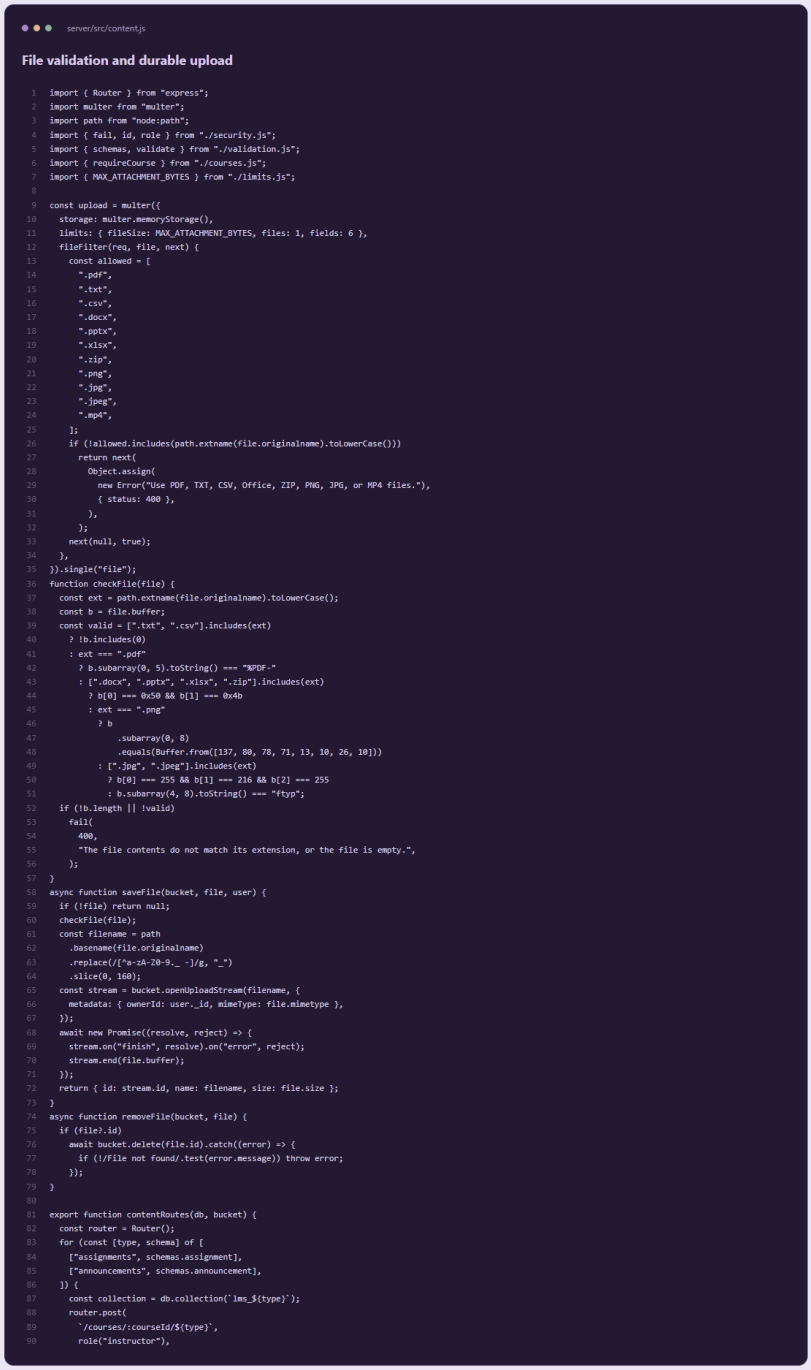
Course ownership and enrollment checks

```
import { Router } from "express";
import { fail, id, role } from "./security.js";
import { schemas, validate } from "./validation.js";

export async function accessCourse(db, user, courseId, owner = false) {
  const course = await db
    .collection("lms_courses")
    .findOne({ _id: id(courseId), deletedAt: { $exists: false } });
  if (!course) fail(404, "Course not found.");
  const isOwner = course.instructorId.equals(user._id);
  if (owner && !isOwner)
    fail(403, "Only the instructor who owns this course can change it.");
  const enrolled =
    user.role === "student" &&
    !!(await db
      .collection("lms_enrollments")
      .findOne({ courseId: course._id, studentId: user._id }));
  return { course, isOwner, enrolled };
}
export async function requireCourse(db, user, courseId, owner = false) {
  const result = await accessCourse(db, user, courseId, owner);
  if (!result.isOwner && !result.enrolled)
    fail(403, "Join this course to access its learning content.");
  return result;
}
export async function visibleCourseIds(db, user) {
  if (user.role === "instructor")
    return (
      await db
        .collection("lms_courses")
        .find({ instructorId: user._id, deletedAt: { $exists: false } })
        .project({ _id: 1 })
        .toArray()
    ).map((c) => c._id);
  const enrolled = await db
    .collection("lms_enrollments")
    .find({ studentId: user._id })
    .toArray();
  return (
    await db
      .collection("lms_courses")
      .find({
        _id: { $in: enrolled.map((e) => e.courseId) },
        deletedAt: { $exists: false },
      })
      .project({ _id: 1 })
      .toArray()
  ).map((c) => c._id);
}
export function courseRoutes(db, bucket) {
  const router = Router();
  const courses = db.collection("lms_courses");
  const enrollments = db.collection("lms_enrollments");
  router.get("/courses", async (req, res) => {
    const page = Math.max(1, Number.parseInt(req.query.page) || 1);
    const limit = 9;
    const search =
      typeof req.query.search === "string"
        ? req.query.search.slice(0, 100).replace(/[.*+?^${}()|[\]\\]/g, "\\$&")
        : "";
    const filter = {
      deletedAt: { $exists: false },
      ...(req.user.role === "instructor"
        ? { instructorId: req.user._id }
        : req.query.scope === "catalog"
          ? { published: true }
          : { _id: { $in: await visibleCourseIds(db, req.user) } }),
    };
    if (search)
      filter.$or = [
        { title: { $regex: search, $options: "i" } },
        { code: { $regex: search, $options: "i" } },
        { category: { $regex: search, $options: "i" } },
```

6. File validation and durable upload

server/src/content.js



7. Protected React routing

client/src/App.jsx

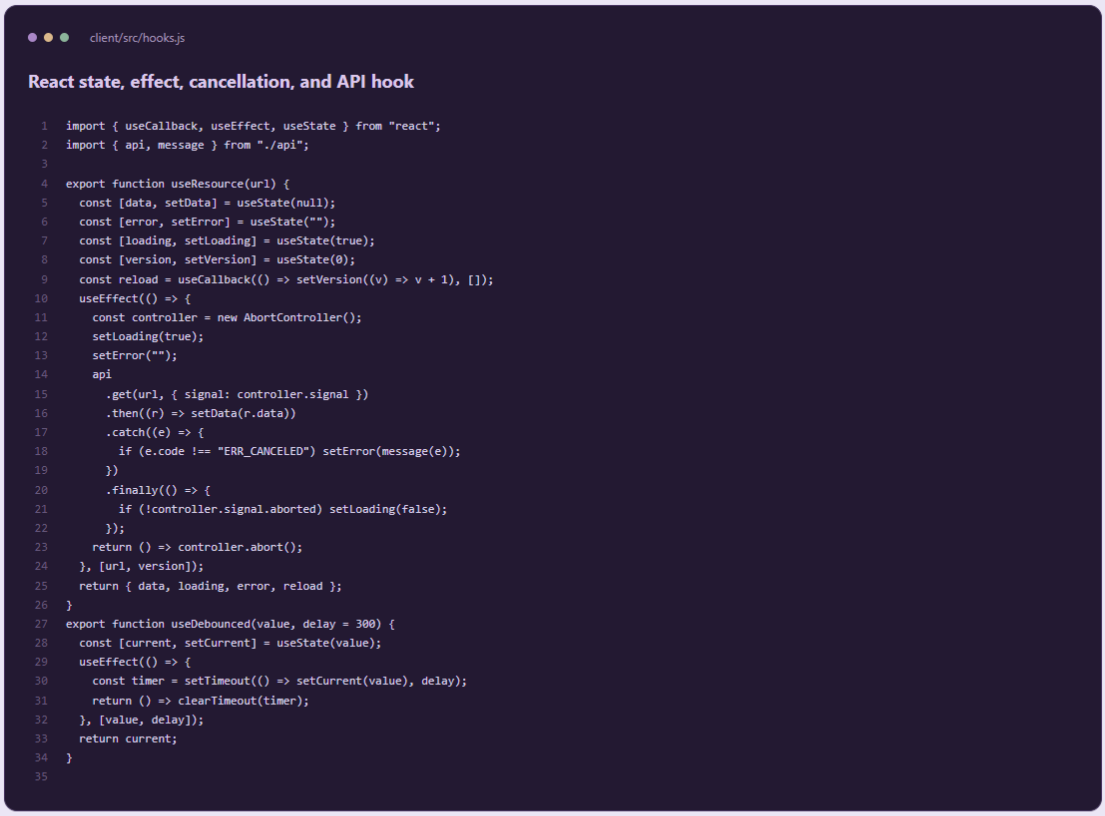
client/src/App.jsx

Protected React routing

```jsx
import {
  BrowserRouter,
  Navigate,
  Outlet,
  Route,
  Routes,
} from "react-router-dom";
import { Toaster } from "react-hot-toast";
import { AuthProvider, useAuth } from "./auth";
import AuthPage from "./AuthPage";
import Shell from "./Shell";
import Dashboard from "./Dashboard";
import Courses from "./Courses";
import CourseDetail from "./CourseDetail";
import ActivityPage from "./ActivityPage";
import Profile from "./Profile";
import { ErrorState, Loading } from "./ui";
import {
  AdminActivity,
  AdminCourses,
  AdminOverview,
  AdminUsers,
} from "./Admin";
import AdminInvitations from "./AdminInvitations";
function Home() {
  const { user } = useAuth();
  return user.role === "admin" ? <AdminOverview /> : <Dashboard />;
}
function AdminOnly() {
  const { user } = useAuth();
  return user.role === "admin" ? <Outlet /> : <Navigate to="/" replace />;
}
function LearningOnly() {
  const { user } = useAuth();
  return user.role !== "admin" ? <Outlet /> : <Navigate to="/" replace />;
}
function Protected() {
  const { user, loading, unavailable, restore } = useAuth();
  if (loading) return <Loading />;
  if (unavailable)
    return (
      <ErrorState
        error="The server is unavailable. Start the API and check your database connection."
        retry={restore}
      />
    );
  return user ? <Shell /> : <Navigate to="/login" replace />;
}
export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<AuthPage key="login" />} />
          <Route
            path="/register"
            element={<AuthPage key="register" register />}
          />
          <Route path="/signup" element={<Navigate to="/register" replace />} />
          <Route element={<Protected />}>
            <Route index element={<Home />} />
            <Route path="admin" element={<AdminOnly />}>
              <Route index element={<Navigate to="/" replace />} />
              <Route path="users" element={<AdminUsers />} />
              <Route path="courses" element={<AdminCourses />} />
              <Route path="activity" element={<AdminActivity />} />
              <Route path="invitations" element={<AdminInvitations />} />
            </Route>
            <Route element={<LearningOnly />}>
              <Route path="/courses" element={<Courses />} />
              <Route path="/courses/:courseId" element={<CourseDetail />} />
              <Route
                path="/assignments"
                element={<ActivityPage type="assignments" />}
              />
              <Route path="/grades" element={<ActivityPage type="grades" />} />
              <Route
                path="/announcements"
                element={<ActivityPage type="announcements" />}
              />
            </Route>
            <Route path="/profile" element={<Profile />} />
            <Route path="*" element={<Navigate to="/" replace />} />
          </Route>
        </Routes>
      </BrowserRouter>
      <Toaster
```

8. React state, effect, cancellation, and API hook

client/src/hooks.js



9. Flutter shared API and student authentication

mobile/lib/api.dart

mobile/lib/api.dart

Flutter shared API and student authentication

```
    'API_BASE_URL',
    defaultValue: kIsWeb
        ? 'http://localhost:8000/api'
        : 'http://10.0.2.2:8000/api',
  );
  final FlutterSecureStorage storage;
  final http.Client client;
  String? _token;
  Json? user;
  LmsApi({FlutterSecureStorage? storage, http.Client? client})
    : storage = storage ?? const FlutterSecureStorage(),
      client = client ?? http.Client();

  Map<String, String> get headers => {
    if (_token != null) 'Authorization': 'Bearer $_token',
  };
  Future<void> restore() async {
    _token = await storage.read(key: 'lms_token');
    if (_token == null) return;
    final data = await get('/auth/me');
    user = Map<String, dynamic>.from(data['user']);
    if (user!['role'] != 'student') {
      await clear();
      throw ApiError('The mobile app is available to students only.');
    }
    notifyListeners();
  }

  Future<void> clear() async {
    _token = null;
    user = null;
    await storage.delete(key: 'lms_token');
    notifyListeners();
  }

  Future<void> authenticate(Json values, {bool register = false}) async {
    final data = await send(
      'POST',
      '/mobile/auth/${register ? 'register' : 'login'}',
      {...values, if (register) 'role': 'student'},
    );
    if (data['user']['role'] != 'student') {
      throw ApiError('Only students can use the mobile app.');
    }
    _token = data['token'];
    await storage.write(key: 'lms_token', value: _token);
    user = Map<String, dynamic>.from(data['user']);
    notifyListeners();
  }

  Future<void> logout() async {
    await send('POST', '/auth/logout');
    await clear();
  }

  Future<dynamic> get(String path) => send('GET', path);
  Future<dynamic> send(String method, String path, [Json? body]) async {
    try {
      final request = http.Request(method, Uri.parse('$baseUrl$path'));
      request.headers.addAll({...headers, 'Content-Type': 'application/json'});
      if (body != null) request.body = jsonEncode(body);
      final streamed = await client
          .send(request)
          .timeout(const Duration(seconds: 30));
      final response = await http.Response.fromStream(
        streamed,
      ).timeout(const Duration(seconds: 30));
      return await _decode(response);
    } on TimeoutException {
      throw ApiError('The server took too long to respond. Please try again.');
```

10. Flutter assignment submission and confirmation

mobile/lib/submission_screen.dart

